

ARRANJO ORDENADO

ALUNO: GUSTAVO RIBEIRO

ArranjoOrdenado

- dados : int[]
- tamanho : int
- crescente : **boolean**

+ ArranjoOrdenado(capacidade: int,
crescente: boolean)

-
- + **inserir**(valor: int) : void
 - + remover(valor: int) : boolean
 - + buscar(valor: int) : int
 - + tamanho() : int
 - + cheio() : boolean
 - + vazio() : boolean

```
if (crescente) {  
    while (i >= 0 && dados[i] > valor) {  
        dados[i + 1] = dados[i];  
        i--;  
    }  
} else {  
    while (i >= 0 && dados[i] < valor) {  
        dados[i + 1] = dados[i];  
        i--;  
    }  
}
```

RESULTADOS OBTIDOS

Inserção	Ordenação crescente	Ordenação decrescente
Inserção em maneira crescente	8.2 +/- 0.30 ms	9.0 +/- 0.41 ms
Inserção em maneira decrescente	12.4 +/- 0.50 ms	7.8 +/- 0.32 ms
Inserção em maneira aleatória	10.1 +/- 0.45 ms	10.3 +/- 0.44 ms

Análise dos Resultados

- **Inserção depende da posição do elemento.**
- **Quando o elemento é inserido no início do vetor, muitos deslocamentos são necessários.**
- **Isso aumenta o tempo de execução.**

Conclusão

- A estrutura `ArranjoOrdenado` possui custo de inserção $O(n)$ no pior caso.
- A ordenação influencia diretamente no desempenho.
- Os experimentos confirmam o comportamento esperado do algoritmo.